



C++ 四级

2025 年 09 月

1 单选题（每题 2 分，共 30 分）

第 1 题 运行下面程序后变量 `a` 的值是（ ）。

```
1 int a = 42;  
2 int* p = &a;  
3 *p = *p + 1;
```

- ☐ A. 42
- ☐ B. 43
- ☐ C. 编译错误
- ☐ D. 不确定

第 2 题 以下关于数组的描述中，（ ）是错误的。

- ☐ A. 数组名是一个指针常量
- ☐ B. 随机访问数组的元素方便快捷
- ☐ C. 数组可以像指针一样进行自增操作
- ☐ D. `sizeof(arr)` 返回的是整个数组 `arr` 占用的字节数

第 3 题 给定如下定义的数组 `arr`，则 `*(*(arr + 1) + 2)` 的值是（ ）。

```
1 int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
```

- ☐ A. 2
- ☐ B. 5
- ☐ C. 4
- ☐ D. 6

第 4 题 下面这段代码会输出（ ）。

```

1  int add(int a, int b = 1); // 函数声明
2
3  int main() {
4      cout << add(2) << " " << add(2, 3);
5      return 0;
6  }
7
8  int add(int a, int b) { // 函数定义
9      return a + b;
10 }

```

- ☐ A. 3 5
- ☐ B. 编译失败: 定义处少了默认参数
- ☐ C. 运行错误
- ☐ D. 链接失败: 未定义引用

第5题 下面这段代码会输出 ()。

```

1  int x = 5;
2
3  void foo() {
4      int x = 10;
5      cout << x << " ";
6  }
7
8  void bar() {
9      cout << x << " ";
10 }
11
12 int main() {
13     foo();
14     bar();
15 }

```

- ☐ A. 5 5
- ☐ B. 10 10
- ☐ C. 5 10
- ☐ D. 10 5

第6题 下面程序运行的结果是 ()。

```

1  void increaseA(int x) {
2      x++;
3  }
4  void increaseB(int* p) {
5      (*p)++;
6  }
7  int main() {
8      int a = 5;
9      increaseA(a);
10     cout << a << " ";
11     increaseB(&a);
12     cout << a;
13 }

```

- ☐ A. 6 7
- ☐ B. 6 6

☐ D. 5 5

第7题 关于结构体初始化，以下哪个选项中正确的是（ ）。

```
1 | struct Point {int x,y};
```

☐ A. Point p = (1,2);

☐ B. Point p = {1,2};

☐ C. Point p = new {1,2};

☐ D. Point p = <1,2>;

第8题 运行如下代码会输出（ ）。

```
1 | struct Cat {
2 |     string name;
3 |     int age;
4 | };
5 |
6 | void birthday(Cat& c) {
7 |     c.age++;
8 | }
9 |
10 | int main() {
11 |     Cat kitty{"Mimi", 2};
12 |     birthday(kitty);
13 |     cout << kitty.name << " " << kitty.age;
14 | }
```

☐ A. Mimi 2

☐ B. Mimi 3

☐ C. kitty 3

☐ D. kitty 2

第9题 关于排序算法的稳定性，以下说法错误的是（ ）。

☐ A. 稳定的排序算法不改变相等元素的相对位置

☐ B. 冒泡排序是稳定的排序算法

☐ C. 选择排序是稳定的排序算法

☐ D. 插入排序是稳定的排序算法

第10题 下面代码试图实现选择排序，使其能对数组 nums 排序为升序，则横线上应分别填写（ ）。

```
1 | void selectionSort(vector<int>& nums) {
2 |     int n = nums.size();
3 |     for (int i = 0; i < n - 1; ++i) {
4 |         int minIndex = i;
5 |         for (int j = i + 1; j < n; ++j) {
6 |             if ( _____ ) { // 在此处填入代码
7 |                 minIndex = j;
8 |             }
9 |         }
10 |         _____; // 在此处填入代码
11 |     }
12 | }
```

☐ A.

```
1 | nums[j] < nums[minIndex]
2 | swap(nums[i], nums[minIndex])
```

☐ B.

```
1 | nums[j] > nums[minIndex]
2 | swap(nums[i], nums[minIndex])
```

☐ C.

```
1 | nums[j] <= nums[minIndex]
2 | swap(nums[j], nums[minIndex])
```

☐ D.

```
1 | nums[j] <= nums[minIndex]
2 | swap(nums[i], nums[j])
```

第 11 题 下面程序实现插入排序（升序排序），则横线上应分别填写（ ）。

```
1 | void insertionSort(int arr[], int n) {
2 |     for (int i = 1; i < n; i++) {
3 |         int key = arr[i];
4 |         int j = i - 1;
5 |         while ( j >= 0 && _____ ) { // 在此处填入代码
6 |             arr[j + 1] = arr[j];
7 |             j--;
8 |         }
9 |         _____; // 在此处填入代码
10 |     }
11 | }
```

☐ A.

```
1 | arr[j] > key
2 | arr[j + 1] = key
```

☐ B.

```
1 | arr[j] < key
2 | arr[j + 1] = key
```

☐ C.

```
1 | arr[j] > key
2 | arr[j] = key
```

☐ D.

```
1 | arr[j] < key
2 | arr[j] = key
```

第 12 题 关于插入排序的时间复杂度，下列说法正确的是（ ）。

- ☐ A. 最好情况和最坏情况的时间复杂度都是 $O(n^2)$
- ☐ B. 最好情况是 $O(n)$ ，最坏情况是 $O(n^2)$
- ☐ C. 最好情况是 $O(n)$ ，最坏情况是 $O(2^n)$
- ☐ D. 最好情况是 $O(n^2)$ ，最坏情况是 $O(2^n)$

第 13 题 小杨正在爬楼梯，需要 n 阶才能到达楼顶，每次可以爬 1 阶或 2 阶，求小杨有多少种不同的方法可以爬到楼顶，横线上应填写（ ）。

```

1  int climbStairs(int n) {
2      if (n <= 2) return n;
3      int prev2 = 1;
4      int prev1 = 2;
5      int current = 0;
6      for (int i = 3; i <= n; ++i) {
7          ----- // 在此处填入代码
8      }
9      return current;
10 }
11

```

☐ A.

```

1  prev2 = prev1;
2  prev1 = current;
3  current = prev1 + prev2;

```

☐ B.

```

1  current = prev1 + prev2;
2  prev2 = prev1;
3  prev1 = current;

```

☐ C.

```

1  current = prev1 + prev2;
2  prev1 = current;
3  prev2 = prev1;

```

☐ D.

```

1  prev1 = current;
2  prev2 = prev1;
3  current = prev1 + prev2;

```

第 14 题 假设有一个班级的成绩单，存储在一个长度为 n 的数组 `scores` 中，每个元素是一个学生的分数。老师想要找出所有满足 `scores[i] + scores[j] + scores[k] == 300` 的三元组，其中 $i < j < k$ 。下面代码实现该功能，请问其时间复杂度是（ ）。

```

1  int cnt = 0;
2  for (int i = 0; i < n; i++) {
3      for (int j = i + 1; j < n; j++) {
4          for (int k = j + 1; k < n; k++) {
5              if (scores[i] + scores[j] + scores[k] == 300) {
6                  cnt++;
7              }
8          }
9      }
10 }
11

```

☐ A. $O(n)$

☐ B. $O(n^2)$

☐ C. $O(n^3)$

☐ D. $O(2^n)$

第 15 题 关于异常处理，以下说法错误的是（ ）。

☐ A. `try` 块中的代码可能会抛出异常

☐ B. `catch` 块可以有多个，处理不同类型的异常

- ☐ C. throw 语句用于抛出异常
- ☐ D. 所有异常都必须被捕获，否则程序会崩溃

2 判断题（每题 2 分，共 20 分）

第 1 题 以下代码能正确初始化指针。

```
1 int a = 5;
2 int *p = a;
```

第 2 题 执行下面C++代码将输出 11 。

```
1 int x = 10;
2 void f() {
3     int x = x + 1;
4     cout << x << endl;
5 }
6
7 int main() {
8     f();
9 }
```

第 3 题 以下C++代码合法。

```
1 struct Student {
2     string name;
3     int age;
4     float score;
5 };
6 Student* students = new Student[20];
```

第 4 题 执行下面C++代码将输出 10 。

```
1 void func(int* p) {
2     *p = 10;
3 }
4
5 int main() {
6     int a = 5;
7     func(&a);
8     cout << a << endl;
9     return 0;
10 }
```

第 5 题 下面代码将二维数组 arr 传递给函数 f，函数内部用 arr[i][j] 访问元素，函数参数声明为 int arr[][4] 是错误的。

```
1 void f(int arr[][4], int rows) {
2     // 访问 arr[i][j]
3 }
4
5 int main() {
6     int arr[3][4] = { /* 初始化 */ };
7     f(arr, 3);
8 }
```

第6题 递推是在给定初始条件下，已知前一项（或前几项）求后一项的过程。

第7题 虽然插入排序的时间复杂度为 $O(n^2)$ ，但由于单元操作相对较少，因此在小数据量的排序任务中非常受欢迎。

第8题 对整数数组 {4, 1, 3, 1, 5, 2} 进行冒泡排序（将最大元素放到最后），执行一轮之后是 {4, 1, 3, 1, 2, 5}。

第9题 以下代码只能捕获 int 类型异常。

```
1 int main() {
2     try {
3         throw 42;
4     } catch (...) {
5         cout << "Caught" << endl;
6     }
7     return 0;
8 }
```

第10题 以下代码将 Hello 写入文件 data.txt。

```
1 ofstream file("data.txt");
2 cout<<"Hello"<< endl;
3 file.close();
```

3 编程题（每题 25 分，共 50 分）

3.1 编程题 1

- 试题名称：排兵布阵
- 时间限制：1.0 s
- 内存限制：512.0 MB

3.1.1 题目描述

作为将军，你自然需要合理地排兵布阵。地图可以视为 n 行 m 列的网格，适合排兵的网格以 1 标注，不适合排兵的网格以 0 标注。现在你需要在地图上选择一个矩形区域排兵，这个矩形区域内不能包含不适合排兵的网格。请问可选择的矩形区域最多能包含多少网格？

3.1.2 输入格式

第一行，两个正整数 n, m ，分别表示地图网格的行数与列数。

接下来 n 行，每行 m 个整数 $a_{i,1}, a_{i,2}, \dots, a_{i,m}$ ，表示各行中的网格是否适合排兵。

3.1.3 输出格式

一行，一个整数，表示适合排兵的矩形区域包含的最大网格数。

3.1.4 样例

3.1.4.1 输入样例 1

```
1 4 3
2 0 1 1
3 1 0 1
4 0 1 1
5 1 1 1
```

3.1.4.2 输出样例 1

```
1 | 4
```

3.1.4.3 输入样例 2

```
1 | 3 5
2 | 1 0 1 0 1
3 | 0 1 0 1 0
4 | 0 1 1 1 0
```

3.1.4.4 输出样例 2

```
1 | 3
```

3.1.5 数据范围

对于所有测试点，保证 $1 \leq n, m \leq 12$, $0 \leq a_{i,j} \leq 1$ 。

3.1.6 参考程序

3.2 编程题 2

- 试题名称：最长连续段
- 时间限制：1.0 s
- 内存限制：512.0 MB

3.2.1 题目描述

对于 k 个整数构成的数组 $[b_1, b_2, \dots, b_k]$ ，如果对 $1 \leq i < k$ 都有 $b_{i+1} = b_i + 1$ ，那么称数组 b 是一个连续段。

给定由 n 个整数构成的数组 $[a_1, a_2, \dots, a_n]$ ，你可以任意重排数组 a 中元素顺序。请问在重排顺序之后， a 所有是连续段的子数组中，最长的子数组长度是多少？

例如，对于数组 $[1, 0, 2, 4]$ ，可以将其重排为 $[4, 0, 1, 2]$ ，有以下 10 个子数组：

$[4], [0], [1], [2], [4, 0], [0, 1], [1, 2], [4, 0, 1], [0, 1, 2], [4, 0, 1, 2]$

其中除 $[4, 0], [4, 0, 1], [4, 0, 1, 2]$ 以外的子数组均是连续段，因此是连续段的子数组中，最长子数组长度为 3。

3.2.2 输入格式

第一行，一个正整数 n ，表示数组长度。

第二行， n 个整数 $a_1, a_2, \dots, a_n$ ，表示数组中的整数。

3.2.3 输出格式

一行，一个整数，表示数组 a 重排顺序后，所有是连续段的子数组的最长长度。

3.2.4 样例

3.2.4.1 输入样例 1

```
1 | 4
2 | 1 0 2 4
```

3.2.4.2 输出样例 1

```
1 | 3
```

3.2.4.3 输入样例 2

```
1 | 9
2 | 9 9 8 2 4 4 3 5 3
```

3.2.4.4 输出样例 2

```
1 | 4
```

3.2.5 数据范围

对于 40% 的测试点，保证 $1 \leq n \leq 8$ 。

对于所有测试点，保证 $1 \leq n \leq 10^5$ ， $-10^9 \leq a_i \leq 10^9$ 。

3.2.6 参考程序